

Exponential Backoff with Jitter, Worked Out by Hand

Why a retrying agent waits 1, 2, 4, 8, ... seconds and not 1, 2, 3, 4 — and why it must wait a *random* amount, every delay counted by hand.

What this document does. It takes the one production failure mode from Module 2.7 — the **B2 model boundary**, where the LLM call returns a 429 rate-limit error and the agent must `with_retry(..., wait_exponential_jitter=True)` — and works the retry schedule out by hand. We fix `base=1s`, `cap=30s` and `stop_after_attempt`, then compute the delay on every attempt, the total elapsed wait, the geometric-series closed form behind it, and the collision math that makes *jitter* non-optional when a whole fleet fails at once. Every number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact. The module's own config — `wait_exponential(min=0.5, max=8)`, `stop_after_attempt(3)` — is the same machine at different constants.

Where this sits. This is **Topic 1 of Module 2.7** (Building Production AI Agents) — the arithmetic under *Pillar 2, Error Boundaries*, boundary B2 (model) and B3 (transport). Its companions: Module 2.7's *Pillar 5* (cost & latency — backoff trades latency for survival) and the *crash-only / recursion_limit* discussion (B4, the agent boundary), where a step cap plays the role `cap` plays here.

Concepts covered, in order: retry delay $d_k = \min(\text{cap}, \text{base} \cdot 2^k)$ · the geometric series $\sum 2^k = 2^r - 1$ · the saturation point where the cap bites · exponential vs. linear backoff under load · the thundering herd · full jitter $\sim \text{Uniform}(0, d_k)$ · collision probability slot/d_k · expected colliders per slot $W \cdot \text{slot}/d_k$.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one client, retrying a rate-limited model call

In Module 2.7 the model boundary wraps every LLM call in a retry policy — `primary.with_retry(` with `stop_after_attempt=3`, `wait_exponential_jitter=True`, and a `retry_if_exception_type` of `RateLimitError` or `APITimeoutError`. When the provider returns 429 Too Many Requests, the agent does not give up and does not hammer the endpoint — it *waits*, then retries, waiting longer each time. The question this document answers is: *how long does it wait, exactly, and why that schedule?*

We index attempts from $k = 0$. Attempt $k = 0$ is the first *retry* after the original call failed; before it, the client sleeps for delay d_0 . The delay on attempt k is

$$d_k = \min(\text{cap}, \text{base} \cdot 2^k). \tag{1}$$

The quantities. We pick round, hand-followable constants. The mechanics are identical at the module's `min=0.5`, `max=8`; only the numbers move.

Quantity	Symbol	Value here
Base delay (the first wait)	base	1 s
Ceiling on any single wait	cap	30 s
Doubling factor per attempt	—	2
Attempt index (0-based)	k	$0, 1, 2, \dots$
Number of retries we tabulate	r	up to 7
Fleet size (thundering-herd case)	W	100 clients
Jitter slot resolution	slot	0.1 s

Notation. d_k is the wait *before* the k -th retry. The cumulative wait after r retries is $S_r = \sum_{k=0}^{r-1} d_k$. “Uncapped” means we drop the min and let $d_k = \text{base} \cdot 2^k$ grow forever; we use it to derive the clean closed form, then put the cap back.

Intuition

Linear backoff ($1, 2, 3, 4, \dots$) treats congestion as if it clears at a steady pace. Exponential backoff assumes the opposite, which is what really happens under load: if the service is still saturated, the smart move is to back off *fast*, doubling the gap so the contending crowd drains before you knock again. The cap stops the doubling from running away to absurd waits — past a point, 30 s is patient enough; 60 s only annoys the user.

2 Step 1 — the delay on each attempt

Plug $k = 0, 1, 2, \dots$ into Equation (1) with base= 1, cap= 30. The raw term $\text{base} \cdot 2^k$ is $1, 2, 4, 8, 16, 32, 64, \dots$. The min clips anything above 30:

Attempt k	raw = 2^k	capped	d_k (s)
0	1	no	1
1	2	no	2
2	4	no	4
3	8	no	8
4	16	no	16
5	32	yes	30
6	64	yes	30

The cap bites the first time $2^k > 30$, i.e. $2^k > 30 \Rightarrow k \geq 5$ (since $2^4 = 16 \leq 30$ but $2^5 = 32 > 30$). From attempt 5 onward every wait is pinned at 30 s. ✓ So the delay sequence is $1, 2, 4, 8, 16, \mathbf{30}, \mathbf{30}$ — doubling until it slams into the ceiling, flat thereafter.

Mini-example — this operation in isolation

Just the doubling, in isolation: each wait is exactly twice the one before, until the cap. $d_0 = 1$, $d_1 = 2 \cdot 1 = 2$, $d_2 = 2 \cdot 2 = 4$, $d_3 = 2 \cdot 4 = 8$, $d_4 = 2 \cdot 8 = 16$. The next would be 32, but $\min(30, 32) = 30$, so $d_5 = 30$ and it stays there. Five clean doublings, then a wall. That single min is the whole difference between “patient” and “a retry that sleeps for

eighteen minutes”.

3 Step 2 — total elapsed wait, and the geometric series

How long has the client waited in total after r retries? Sum the delays. Ignore the cap for a moment (it has not bitten yet for small r) so $d_k = \text{base} \cdot 2^k$:

$$S_r = \sum_{k=0}^{r-1} \text{base} \cdot 2^k = \text{base} \sum_{k=0}^{r-1} 2^k. \quad (2)$$

The inner sum is a finite geometric series with ratio 2. Derive its closed form the standard way — multiply by the ratio and subtract. Let $G = \sum_{k=0}^{r-1} 2^k = 1 + 2 + 4 + \dots + 2^{r-1}$. Then $2G = 2 + 4 + \dots + 2^{r-1} + 2^r$, and subtracting,

$$2G - G = 2^r - 1 \quad \implies \quad G = \sum_{k=0}^{r-1} 2^k = 2^r - 1.$$

So the uncapped total wait has the clean closed form

$$S_r^{\text{uncapped}} = \text{base} (2^r - 1) \quad (3)$$

With $\text{base} = 1$ this is just $2^r - 1$. Check it against the running sum of the (capped) delays from Step 1, and watch exactly where the two part company:

retries r	last d_{r-1}	cumulative S_r	$2^r - 1$	match?
1	1	1	1	✓
2	2	3	3	✓
3	4	7	7	✓
4	8	15	15	✓
5	16	31	31	✓
6	30	61	63	cap bites
7	30	91	127	cap bites

The closed form $2^r - 1$ tracks the real cumulative wait *exactly* through $r = 5$ (1, 3, 7, 15, 31). At $r = 6$ the sixth delay would have been 32 but was capped to 30, so the real total is $31 + 30 = 61$ while the uncapped formula says 63 — a 2 s gap, exactly the $32 - 30$ we clipped. Past the cap, the total grows *linearly* at 30 s per retry, not exponentially. ✓

Intuition

Two regimes live in one schedule. **Before the cap**, total wait is $\approx$ the last delay — because $1 + 2 + 4 + \dots + 2^{r-1} = 2^r - 1$ is just under $2^r = 2 \cdot 2^{r-1}$, i.e. a hair under twice the final term. So “how long have I waited in total” is roughly “how long was my last sleep”, doubled. **After the cap**, each retry adds a flat **cap**, so total wait becomes a straight line. The cap converts an explosive curve into a polite arithmetic one.

4 Step 3 — why exponential, not linear

Why double, instead of adding a constant? Consider the failure that triggers retries in the first place: the service is *overloaded*. Suppose N clients are all retrying.

- **Linear backoff** ($d_k = \text{base} \cdot (k+1)$): waits 1, 2, 3, 4, 5, ... The inter-retry gap is *constant* (+1 s each time). Under sustained overload the crowd keeps knocking at a steady rate — you relieve pressure slowly, and the queue may never drain faster than it fills.
- **Exponential backoff** ($d_k = \text{base} \cdot 2^k$): waits 1, 2, 4, 8, 16, ... The gap *doubles*. After only 5 retries the crowd is knocking 16× less often than at the start; the contention drains geometrically. This is the regime that lets an overloaded service actually recover.

Quantitatively, to reach a 16 s spacing, linear backoff needs 16 retries (and has by then waited $\sum_{j=1}^{16} j = 136$ s); exponential reaches it in 5 retries having waited only 31 s. Exponential buys a large spacing for a small total wait — the geometric series $2^r - 1$ doing its work in reverse. ✓

Watch out

Backoff trades *latency* for *survival*, and that trade has a ceiling set elsewhere. Module 2.7 wraps the whole agent in `asyncio.wait_for(..., timeout=90)` (boundary B4). If your retries are $1 + 2 + 4 + 8 + 16 + 30 = 61$ s of pure *sleeping*, you have spent two-thirds of a 90 s request budget waiting before the model has even answered once. Backoff schedules and outer timeouts must be designed *together*: `stop_after_attempt` and `cap` exist precisely so the sleep total cannot blow the request deadline.

5 Step 4 — the thundering herd, and why jitter

Now the part that makes jitter mandatory. Imagine the rate limiter trips for a whole *fleet*: $W = 100$ clients all get 429 at the same instant $t = 0$. With the *deterministic* schedule above, every one of them computes the identical $d_0 = 1$ s and retries at *exactly* $t = 1$. All 100 collide again — a synchronized spike that re-trips the limiter, and they collide again at $t = 3$, $t = 7$, ... This is the **thundering herd**: backoff alone just moves the stampede, it does not disperse it.

Full jitter fixes this by randomizing each wait. Instead of sleeping exactly d_k , each client samples its delay uniformly:

$$\text{delay} \sim \text{Uniform}(0, d_k). \quad (4)$$

Now the 100 retries are smeared across the whole interval $[0, d_k]$ instead of stacking on one instant. Quantify the relief. Chop the interval into slots of width $\text{slot} = 0.1$ s. With $d_5 = 30$ s there are $30/0.1 = 300$ slots. Each client lands in one slot, chosen uniformly, so the probability a given client lands in a particular slot is $\text{slot}/d_k = 0.1/30 \approx 0.00333$. The expected number of clients per slot is

$$\mathbb{E}[\text{clients per slot}] = W \cdot \frac{\text{slot}}{d_k} = 100 \cdot \frac{0.1}{30} = \frac{100}{300} \approx 0.333. \quad (5)$$

So the load drops from **100 clients in one instant** (no jitter) to ≈ 0.33 **clients per 100 ms slot** (full jitter) — a 300× reduction in peak concurrency. ✓ Equivalently, for any fixed client the expected number of *others* sharing its slot is $(W - 1) \cdot \text{slot}/d_k = 99 \cdot 0.00333 \approx 0.33$: most clients now retry alone.

Mini-example — this operation in isolation

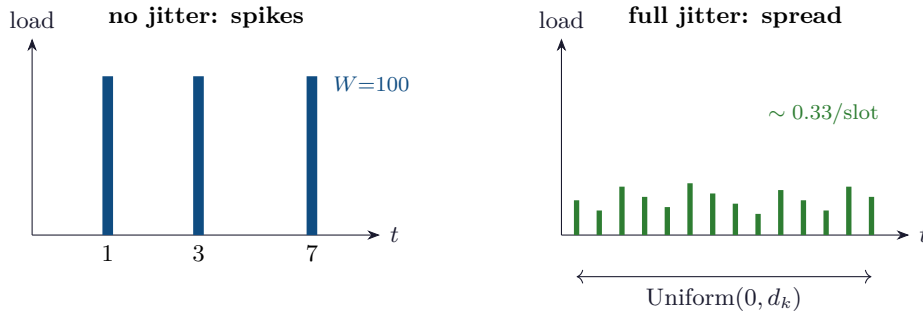
The collision probability for two specific clients landing in the same 0.1 s slot of a 30 s window is $\text{slot}/d_k = 0.1/30 \approx 0.0033$, about one in 300. Without jitter that probability is 1 — they are guaranteed to collide, because both sleep exactly 1 s, then exactly 2 s, forever in lockstep. Jitter turns a certainty into a 0.3% chance. A quick simulation of 100 clients dropped uniformly into 300 slots finds, on average, only about 28 of the 100 share a slot with *anyone* — versus all 100 colliding without it.

Intuition

Backoff and jitter solve two different halves of the same problem. **Backoff** (the doubling) controls *how often a single client knocks* — it drains one client's pressure over time. **Jitter** (the randomization) controls *whether many clients knock together* — it desynchronizes the fleet so their retries never line up. You need both: exponential backoff without jitter just produces well-spaced *stampedes*; jitter without backoff produces a constant random drizzle that never backs off. Production code (the module's `wait_exponential_jitter=True`) always combines them.

6 The schedule, drawn

Synchronized retries (left) all fire at the same instants and stack into spikes; full-jitter retries (right) spread across each interval, flattening the load.



Same total number of retries on both sides; jitter only changes *when* each fires. The left stacks 100 into three spikes; the right scatters them so no slot holds more than about one. The picture is Equation (5).

7 Backoff cheat-sheet

Delay on attempt k	$d_k = \min(\text{cap}, \text{base} \cdot 2^k)$
Uncapped cumulative wait	$S_r = \text{base} (2^r - 1)$
Geometric-series identity	$\sum_{k=0}^{r-1} 2^k = 2^r - 1$
Cap bites when	$\text{base} \cdot 2^k > \text{cap}$
Post-cap total (linear)	$S_r \approx S_{k^*} + (r - k^*) \text{cap}$
Full jitter	$\text{delay} \sim \text{Uniform}(0, d_k)$
Expected colliders per slot	$W \cdot \frac{\text{slot}}{d_k}$
Sync vs. jitter (our numbers)	$100 \rightarrow \approx 0.33 \text{ per } 100\text{ms slot}$

8 An honest word about these numbers

The constants here (base= 1 s, cap= 30 s, $W = 100$, slot= 100 ms) are illustrative round numbers, chosen so the doublings and the $2^r - 1$ sum are followable on paper — the module's own policy uses `min=0.5`, `max=8`, `stop_after_attempt=3`, and a real service might cap at 60 s with W in the thousands. What is *not* illustrative is the structure. The delay is always $\min(\text{cap}, \text{base} \cdot \text{factor}^k)$; the uncapped total is always the geometric closed form (for factor 2, exactly $2^r - 1$); the cap always converts the tail from exponential to linear; and full jitter always reduces expected peak concurrency by the factor slot/d_k , turning a guaranteed collision into a slot/d_k chance. Change the constants and the seconds move; the shape — doubling, cap, jitter — does not. That shape is exactly what `wait_exponential_jitter=True` encodes in one flag, and why no production retry policy ships without it.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the per-attempt delays, the cumulative-wait table, the geometric-series check, and the thundering-herd / jitter arithmetic; change `base`, `cap`, `W`, or `slot` to explore your own retry policy.

```

1  # backoff.py -- reproduces every number in "Exponential Backoff with Jitter".
2  import random
3  from collections import Counter
4
5  base = 1.0 # first wait, seconds
6  cap = 30.0 # ceiling on any single wait, seconds
7
8  def delay(k): # d_k = min(cap, base * 2**k)
9      return min(cap, base * (2 ** k))
10
11 # --- per-attempt delays and cumulative wait (attempts 0..6) ---
12 cum = 0.0
13 for k in range(7):
14     cum += delay(k)
15     uncapped = base * (2 ** (k + 1) - 1) # closed form for r = k+1 retries
16     print(f"k={k} d_k={delay(k):4.0f} cum={cum:4.0f} 2^r-1={uncapped:4.0f}")
17
18 # --- geometric series closed form: sum_{k=0}^{r-1} 2^k = 2^r - 1 ---
19 for r in range(1, 7):

```

```

20     s = sum(base * (2 ** k) for k in range(r))
21     assert abs(s - base * (2 ** r - 1)) < 1e-9
22     print("geometric identity 2^r-1 verified for r=1..6")
23
24     # --- thundering herd / full jitter ---
25     W, window, slot = 100, 30.0, 0.1 # window = d_5 = cap
26     n_slots = window / slot # 300
27     exp_per_slot = W / n_slots # 0.333...
28     p_same = slot / window # 0.00333...
29     print(f"no jitter : {W} clients collide in 1 instant")
30     print(f"jitter : E[clients/slot] = W*slot/window = {exp_per_slot:.3f}")
31     print(f"P(two share a slot) = slot/window = {p_same:.5f}")
32
33     # --- Monte Carlo: how many of W share a slot with anyone (full jitter) ---
34     random.seed(42)
35     trials, total = 20000, 0
36     for _ in range(trials):
37         slots = [int(random.random() * n_slots) for _ in range(W)]
38         c = Counter(slots)
39         total += sum(v for v in c.values() if v >= 2)
40     print(f"MonteCarlo: avg {total/trials:.0f} of {W} collide WITH jitter "
41           f"(vs {W} without)")
42
43     # Expected output:
44     # k=0 d_k= 1 cum= 1 2^r-1= 1
45     # k=1 d_k= 2 cum= 3 2^r-1= 3
46     # k=2 d_k= 4 cum= 7 2^r-1= 7
47     # k=3 d_k= 8 cum= 15 2^r-1= 15
48     # k=4 d_k= 16 cum= 31 2^r-1= 31
49     # k=5 d_k= 30 cum= 61 2^r-1= 63
50     # k=6 d_k= 30 cum= 91 2^r-1= 127
51     # geometric identity 2^r-1 verified for r=1..6
52     # no jitter : 100 clients collide in 1 instant
53     # jitter : E[clients/slot] = W*slot/window = 0.333
54     # P(two share a slot) = slot/window = 0.00333
55     # MonteCarlo: avg 28 of 100 collide WITH jitter (vs 100 without)

```

— end of document —